NEON POKER · БЭКЕНД ОНЛАЙН-ПОКЕРА

Протокол клиент ↔ сервер

Всё общение во время игры идёт по одному WebSocket-соединению. HTTP нужен только чтобы получить токен и посмотреть состояние сервера.

Конверт

Одинаковый в обе стороны:

```
{ "t": "тип_сообщения", "d": { }, "rid": "необязательный-id-запроса" }
```

- `t` — тип сообщения;
- `d` — полезная нагрузка;
- `rid` — клиент проставляет сам; сервер вернёт его в ответе или в `error`, чтобы можно было связать ответ с запросом.

Все суммы — **целые фишки**. Дробей нет нигде. Карты — строки из двух символов: ранг (`2..9 T J Q K A`) + масть (`c d h s`), например `As` , `Td` , `7c` .

Подключение

1. `POST /auth/guest {"nickname": "Женя"}` → `{"token": "...", "player_id": "...", "balance": 10000}`
2. `ws://host:8000/ws?token=<token>`

Токен гостя живёт 30 суток и служит ключом от аккаунта — клиенту стоит сохранить его на устройстве. Чтобы продлить ту же личность (и не потерять баланс), пришлите свой прошлый токен: `POST /auth/guest {"nickname": "Женя", "token": "<старый>"}` . Передать просто `player_id` нельзя: он публичен — соперники видят его в составе стола, и по нему кто угодно выпустил бы себе токен на чужой аккаунт.

Токен можно не класть в query, а прислать первым сообщением: `{"t": "auth", "d": {"token": "..."}}` . До авторизации остальные сообщения не принимаются.

Сразу после подключения сервер шлёт `welcome` . Если у игрока уже есть место за столом (реконнект), следом придёт `table_snapshot` .

Клиент → сервер

`t`	`d`	**Что делает**
`auth`	`{token}`	Авторизация, если токена не было в query
`ping`	`{}`	Проверка живости, в ответ `pong`
`list_stakes`	`{}`	Список уровней ставок и статистика столов
`quick_join`	`{stake?, buy_in?}`	**Автоподбор**: сервер сам найдёт стол со свободным местом или создаст новый
`action`	`{hand_id?, action, amount}`	Ход в раздаче
`leave_table`	`{}`	Встать из-за стола (в середине раздачи — после её окончания)
`sit_out`	`{}`	Пропускать раздачи, место сохраняется
`sit_in`	`{}`	Вернуться в игру
`rebuy`	`{amount}`	Докупить фишек с баланса (во время раздачи вступит в силу со следующей)
`request_snapshot`	`{}`	Запросить полное состояние стола
`chat`	`{text}`	Сообщение в чат стола

`action` подробно

```
{ "t": "action", "d": { "hand_id": "low-a1b2-7-x", "action": "raise", "amount": 120 } }
```

- `action`: `fold` | `check` | `call` | `bet` | `raise` | `all_in`
- `amount` — **суммарная ставка «до»** на этой улице, а не приращение. Если текущая ставка 40 и вы хотите поднять до 120, шлёте `amount: 120`.
- Для `fold` / `check` / `call` / `all_in` поле `amount` игнорируется.
- `hand_id` необязателен, но лучше присылать: сервер отклонит ход, опоздавший к уже закончившейся раздаче.

Допустимые значения всегда приходят в `action_required.options` — клиенту не нужно самому считать минимальный рейз.

Сервер → клиент

Служебные

`t`	Когда	Содержимое
`welcome`	сразу после подключения	`player`, `stakes`, `max_seats`, `action_timeout`, `server_time`
`pong`	ответ на `ping`	`server_time`
`stakes`	ответ на `list_stakes`	`stakes`, `tables`
`error`	при любой ошибке	`code`, `message`

Коды ошибок: `unauthorized`, `bad_message`, `not_at_table`, `already_at_table`, `join_in_progress`, `unknown_stake`, `insufficient_funds`, `invalid_buy_in`, `invalid_action`, `not_your_turn`, `rate_limited`, `internal`.

`join_in_progress` — предыдущий `quick_join` этого же игрока ещё выполняется (например, открыли игру во второй вкладке). Повторять запрос не нужно: дождитесь `joined_table` по первому.

Стол

`t`	Когда
`joined_table`	вы сели за стол; содержит полный снимок
`table_snapshot`	реконнект или `request_snapshot`; полный снимок
`left_table`	вы встали; `returned` — сколько фишек вернулось на баланс
`player_joined` / `player_left`	кто-то сел / встал
`seat_update`	изменилось состояние места (сит-аут, реконнект, докупка)
`table_state`	стол ждёт игроков / идёт раздача
`chat`	сообщение в чате

Снимок стола:

```
{
  "table":  { "table_id": "low-a1b2c3d4", "stake": {...}, "max_seats": 6,
              "state": "hand_in_progress", "button_seat": 2, "hand_no": 41 },
  "seats":  [ { "seat_no": 0, "player_id": "guest-…", "nickname": "Женя",
                "stack": 940, "state": "in_hand", "bet": 20, "committed": 20,
                "connected": true, "sitting_out": false, "has_cards": true }, … ],
  "hand":   { "hand_id": "…", "street": "flop", "board": ["As","Kd","7c"],
              "pot": 130, "current_bet": 20, "min_raise": 20, "to_act": 3,
              "deadline_in": 12.4, "deck_id": 3, "deck_sequence": 118 },
  "you":    { "seat_no": 0, "stack": 940, "hole_cards": ["Qh","Qs"],
              "balance": 8600, "options": [ … ] },
  "server_time": 1786000000.0
}
```

`hole_cards` и `options` приходят только в поле `you` — чужие карты сервер не отправляет никогда, даже в зашифрованном виде.

Раздача

`t`	**Кому**	**Содержимое**
`hand_started`	всем	`hand_id`, `button_seat`, места блайндов, `deck_id`, `deck_sequence`, состав игроков
`blind_posted`	всем	`seat_no`, `kind` (`small_blind` / `big_blind`), `amount`, `pot`
`hole_cards`	**только владельцу**	`seat_no`, `cards`
`cards_dealt`	всем	порядок мест, которым раздали (для анимации)
`action_required`	ходящему — полный, остальным — урезанный	см. ниже
`player_action`	всем	`seat_no`, `action`, `amount`, `bet`, `stack`, `pot`
`action_timeout`	всем	время вышло, сервер сходил сам (`check` или `fold`)
`street`	всем	`street`, `cards` (новые), `board` (весь), `pot`
`uncalled_bet_returned`	всем	излишек ставки вернулся игроку
`showdown`	всем	вскрытые руки: `cards`, `combination`, `best_five`
`hand_finished`	всем	`board`, `total_pot`, `pots`, `winners`, `stacks`

`action_required` ходящему игроку:

```
{
  "t": "action_required",
  "d": {
    "seat_no": 3, "hand_id": "…", "street": "flop",
    "to_call": 40, "current_bet": 40, "min_raise": 40, "pot": 130, "stack": 860,
    "time_limit": 20.0, "deadline_ts": 1786000020.0,
    "options": [
      {"action": "fold"},
      {"action": "call",   "min_amount": 40,  "max_amount": 40},
      {"action": "raise",  "min_amount": 80,  "max_amount": 860},
      {"action": "all_in", "min_amount": 860, "max_amount": 860}
    ]
  }
}
```

Остальным за столом уходит то же сообщение без `options` и `stack` — чтобы клиент нарисовал таймер над нужным местом.

`deadline_ts` — абсолютное серверное время (unix, секунды). Клиенту надёжнее считать остаток как `deadline_ts - server_time_из_welcome - прошедшее локально`, чем полагаться на локальные часы.

Таймер хода и авто-фолд

- На решение даётся `action_timeout` секунд (по умолчанию 20).
- Если игрок в дисконнекте — плюс `disconnect_grace` секунд.
- По истечении сервер ходит сам: `check`, если это возможно, иначе `fold`, и рассылает `action_timeout`.
- После `AUTO_FOLD_LIMIT` авто-фолдов подряд игрока переводят в сит-аут — чтобы «уснувший» не тормозил стол. Вернуться: `sit_in`.

Реконнект

Место закрепляется за игроком на `RECONNECT_WINDOW_SECONDS` (по умолчанию 120 с).

1. Клиент переподключается с тем же `player_id` (токен можно переиспользовать, пока не истёк, либо получить новый, передав `player_id` в `/auth/guest`).
2. Сервер шлёт `welcome`, затем `table_snapshot` — со своими карманными картами, бордом, банком и остатком времени на ход.
3. Раздача всё это время продолжается: если подошла очередь, а игрока нет, сработает авто-фолд.

Если игрок не вернулся за отведённое окно — место освобождается, стек возвращается на баланс.

Одновременно у игрока может быть только одно активное соединение. Второй коннект с тем же `player_id` вытесняет первый (код закрытия `4001`), место за столом при этом не теряется.

Честность раздачи

Колоды не генерируются в рантайме. Они лежат в `data/decks.json`, а сервер берёт их по кругу по глобальному счётчику в Redis (`INCR`). В каждом `hand_started` приходят `deck_id` и `deck_sequence` — по ним раздачу можно воспроизвести и проверить постфактум.

`GET /decks` отдаёт контрольные суммы всех колод и текущее положение счётчика. Сами карты через API не отдаются: иначе, зная позицию счётчика, клиент вычислил бы будущие раздачи.